TECHNICAL REPORT · V0.2.17

# Jinbao: A Local-First Desktop Game Coach

*Turning a game client's own telemetry into one actionable sentence — and what breaks when you try*

**Yuan Cao**

Jinbao Project · github.com/YuanCao1996/dsh-game-jinbao

November 2026 · MIT License · Reference implementation: Windows desktop client, ~23k LOC

**ABSTRACT**

Jinbao is a Windows desktop coach that reads a game client's local telemetry (log streams and the LCU API), projects it into a small set of *facts*, decides whether the current moment is worth interrupting the player, and — only then — asks a language model for one or two actionable sentences that it renders in an always-on-top overlay and optionally speaks.

The engineering problem is not "call a model with game state". It is **(i) deciding when to speak**, because a coach that narrates every event is worse than silence; **(ii) not lying**, because speculation must never be described as fact and OCR-based observation must never guess a choice into the permanent record; and **(iii) staying correct across time**, because advice generated forty seconds ago may be actively harmful now.

本文记录这套系统的架构与取舍：事实投影、播报时机与准入控制、强化识别的确认式识别策略、 建议有效性与依赖指纹、签名更新与本地优先的数据模型，以及三个真实故障——它们比任何功能都更能说明 「让模型看懂游戏」难在哪里。

## 1. Problem Framing

Players already have statistics: win rates, tier lists, build sites. The questions that actually stop a player mid-match are situational and short-lived:

- *"That fight just went wrong — why?"*
- *"I have 1200 gold and two magic-damage threats. What do I buy right now?"*
- *"Three augments are offered; which one fits the two I already own?"*

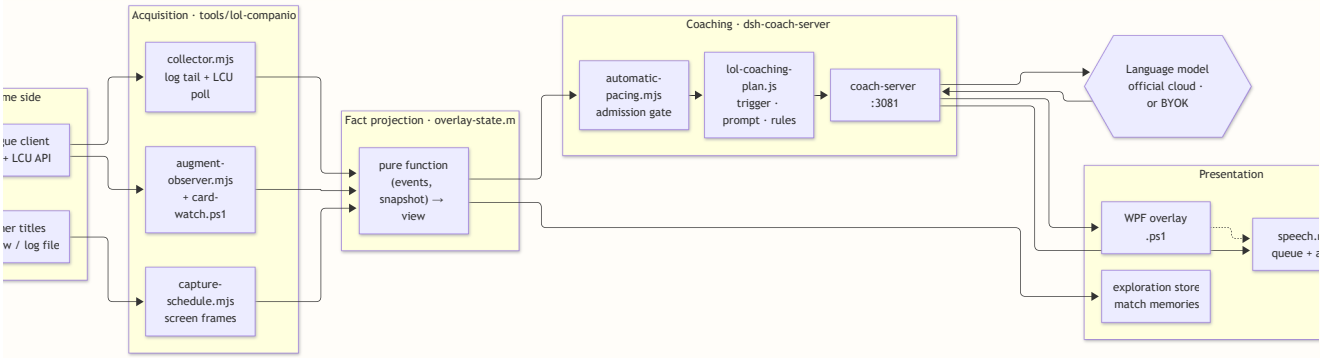
Each answer is worthless twenty seconds late, must name concrete entities (a champion, an item, an augment), and must be executable. That defines the design constraints in Table 1.

**Table 1.** Constraints derived from the framing, and their architectural consequences.

| Constraint | Consequence |
|---|---|
| Advice expires | Every recommendation carries explicit dependencies and a TTL |
| Advice must be grounded | Facts and speculation are separated inside the prompt itself |
| Interruptions are costly | A single admission gate decides what may speak |
| Data is intimate | All match data stays local; exports are redacted |

## 2. System Overview

Four layers, one direction of data flow (Figure 1). Acquisition produces an append-only event journal; projection is a pure function from journal plus snapshot to a view model; the coaching layer decides whether to speak and what to say; presentation renders and optionally speaks it.




1. System context. Acquisition never mutates history; projection is pure; the coaching layer owns every decision about interrupting the play

**Table 2.** Component responsibilities and the invariant each layer must preserve.

| Layer | Directory | Invariant |
|---|---|---|
| Acquisition | `tools/lol-companion/` | Observations are append-only; nothing is rewritten after the fact |
| Projection | `overlay-state.mjs` | Pure: same events in, same view out |
| Coaching | `tools/dsh-coach-server/` | Facts and speculation are separated *in the prompt* |
| Presentation | `tools/desktop-release/` | Overlay failure must never kill the match pipeline |
| Delivery | `tools/jinbao-launcher/` | Ship only what the signature verifies |

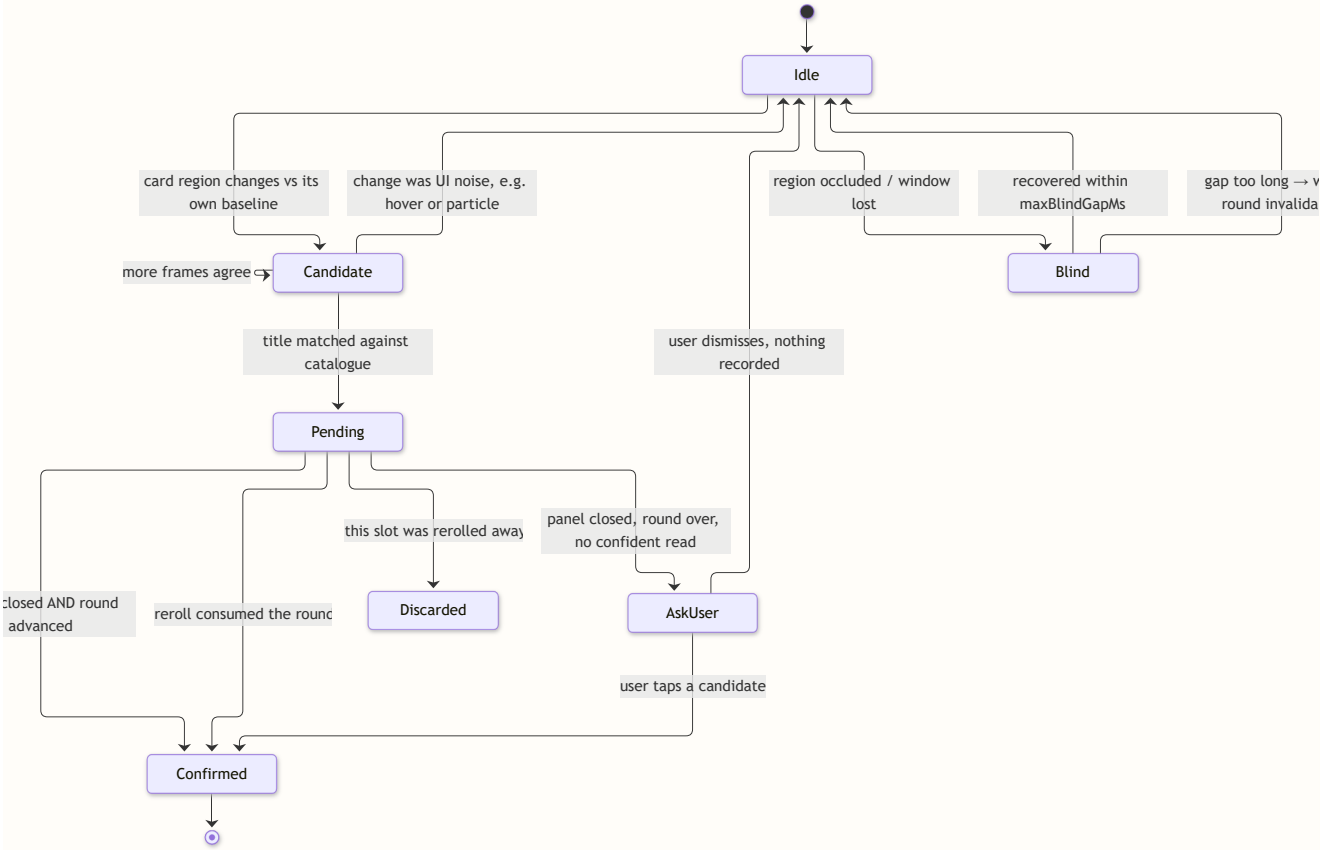
## 3. Acquisition

### 3.1 Log streams and the LCU API

The League client writes a per-match event journal containing kills, items, objectives, phase changes and raw client snapshots. A collector tails it and publishes normalized events; a second stream polls the LCU ( `/liveclientdata/*` ) for authoritative player state. Both streams are append-only and timestamped.

### 3.2 Augment observation is the hard part

Augment identity is not exposed by the API, so it must be read off three card regions on screen. Naive implementations fail here in a specific way: a wrong guess is *permanent*, because it enters the match record and therefore every later recommendation. Our approach trades recall for precision.




2. Augment observation as an explicit state machine. A pending pick is settled by evidence — panel close, round change, reroll — never by t optimism.

**Table 3.** Guards around augment observation, and the failure each one prevents.

| Guard | Value | Why it exists |
|---|---|---|
| Frame freshness | ≤ 4 s | A stale frame must not confirm a live round |
| Blind-gap tolerance | 15 s | Beyond this, earlier readings are untrustworthy |
| Punctuation normalization | applied | OCR read `质变．棱彩阶` where the catalogue says `质变：棱彩阶`, silently dropping a card |
| Title joining | same card only | Titles wrap across lines; description lines must never be fuzzy-matched |
| Fallback | confirmation panel | A wrong fact poisons every later recommendation |

## 4. Fact Projection

`overlay-state.mjs` is a pure function of (events, snapshots) to a view model. Two properties matter. The displayed clock comes from the game's elapsed seconds, never from observation time — mixing the two makes

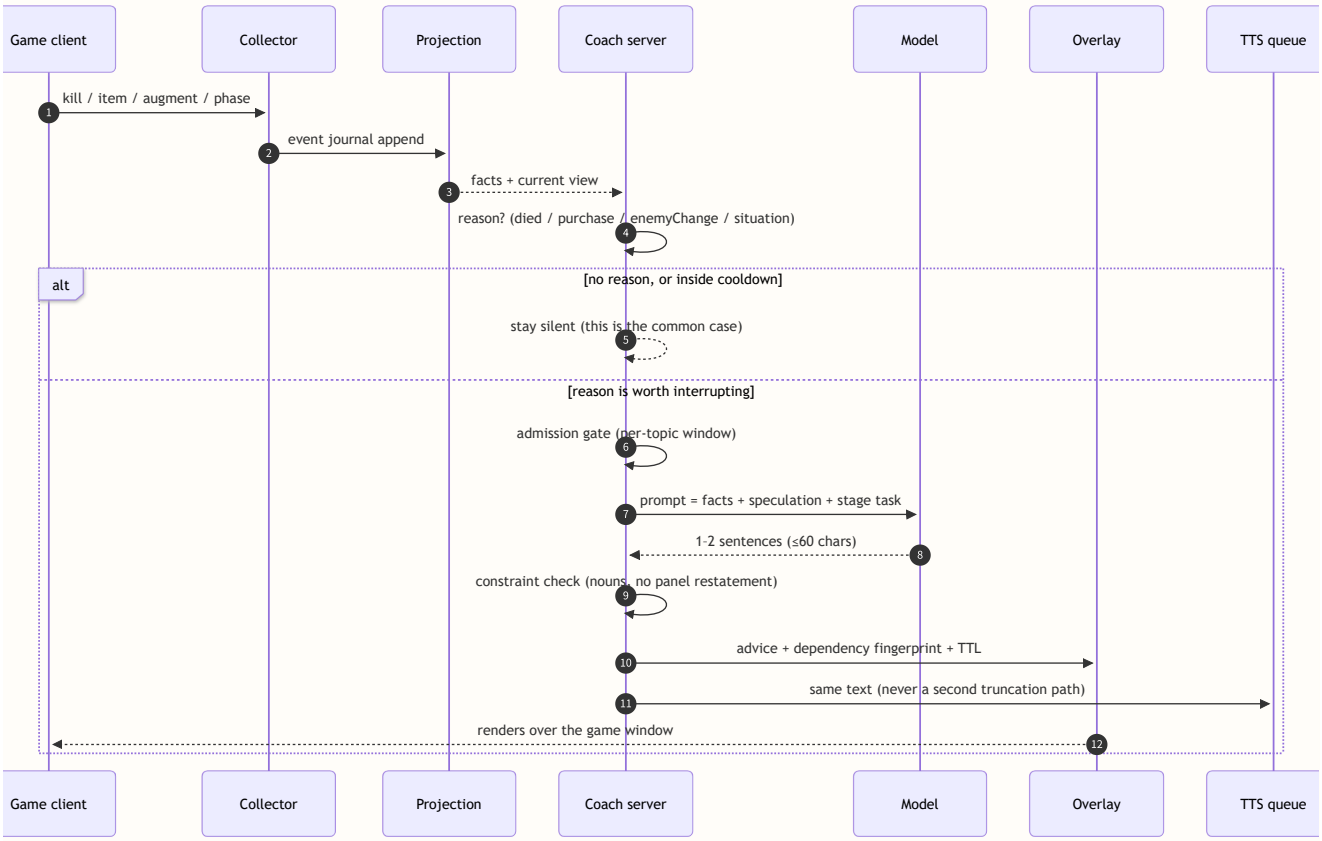
advice say "at 12:30" about a 09:10 event. And projected state is bounded: the key-event page keeps at most forty rows and the panel two hundred, so a long match cannot grow prompt size without limit.

> **Boundary note.** The projected view is deliberately presentation-shaped: `events` and `augments` are rendered text, not arrays. That boundary is useful, and it is also the source of the worst regression in this project (§8.3).

## 5. Coaching Decision Layer

### 5.1 Speak only when it changes a decision

Reasons are computed from deltas rather than from event presence, and each reason carries its own cooldown: death 15 s, material change 75 s, enemy change 30 s. Ordinary minion kills are context, not triggers. The failure mode avoided here is a coach narrating the scoreboard.




3. One advice turn. The upper branch — no reason, or inside a cooldown — is the common case by design.

### 5.2 Admission control

Battle advice and background build analysis share one permit but track *per-topic* windows. An earlier single shared window let the far more frequent background tasks starve advice: in a real match we measured 20 `output_backpressure` skips in twelve minutes with `advicePublished: 0`. Per-topic tracking fixed it.

### 5.3 Prompt construction

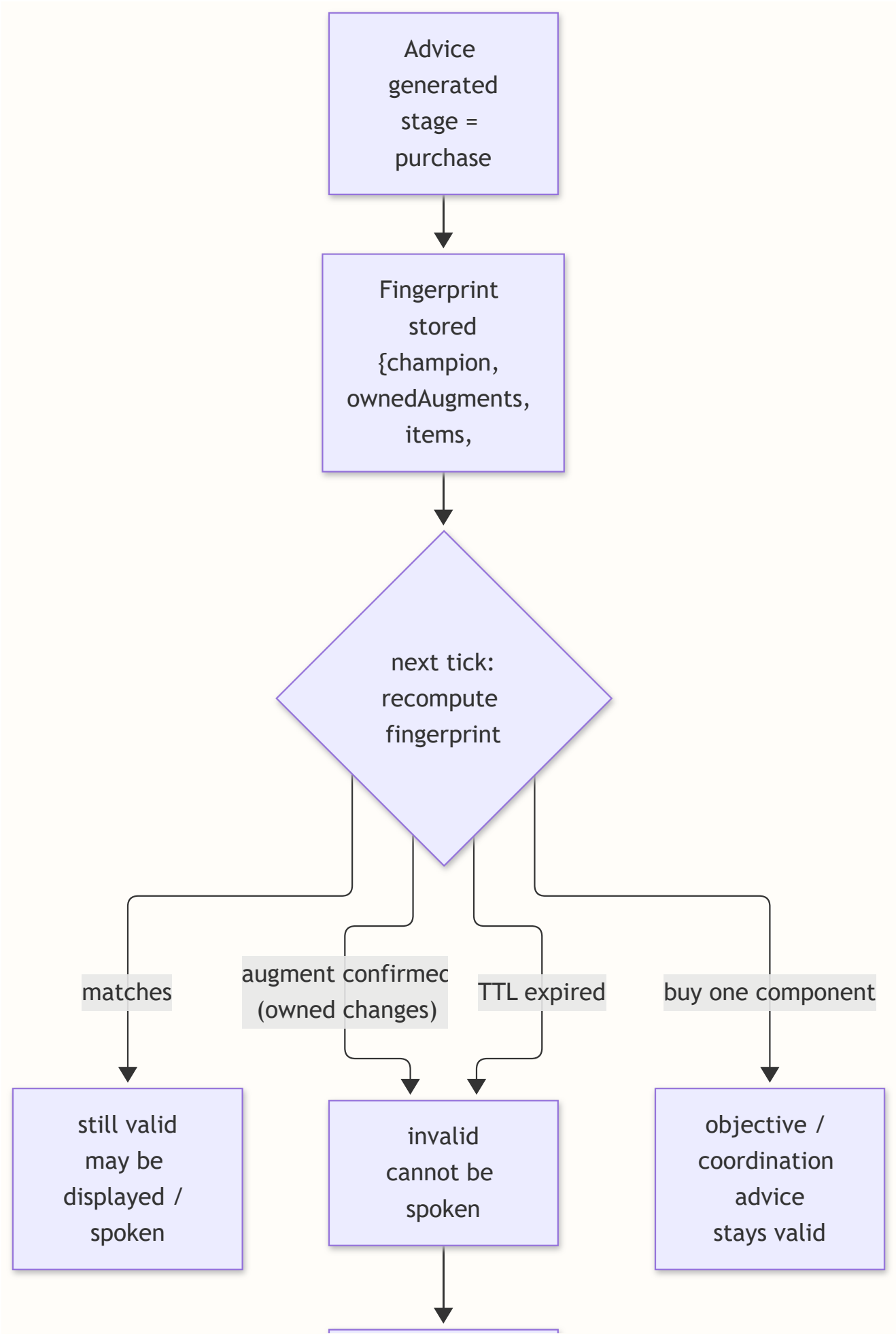
- **Speculation is labelled in the prompt**, not left to the model's judgement: unconfirmed augments are passed as "consider … pending confirmation" candidates and may never be described as owned.
- **Hard fields win.** Multikill counts come from a single field derived from the game's `KillStreak`, because models reliably miscount kills.
- **No panel restatement.** The interface already shows the ranking; the answer must be a decision.
- **Concrete nouns.** A death turn must name one purchasable item; "buy some defence" is rejected by rule.

### 5.4 Output constraints

At most two sentences and sixty characters in total, one paragraph; urgent situations get one sentence. Speech and display share the same source text — a separate truncation path used to cut mid-item ("亡者的板甲，针"), so display now strips sources from the already-final text and speech keeps its own explicit limit.

### 5.5 Validity and invalidation

Advice is not "valid until sixty seconds pass". Each recommendation records what it depends on, and validity is recomputed by comparing fingerprints (Figure 4).


Advice generated stage = purchase
Fingerprint stored {champion, ownedAugments, items,
next tick: recompute fingerprint
matches
augment confirmed (owned changes)
TTL expired
buy one component
still valid may be displayed / spoken
invalid cannot be spoken
objective / coordination advice stays valid

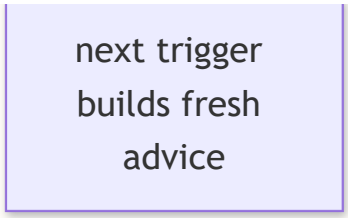



**Figure 4.** Invalidation by dependency fingerprint. A new confirmed augment revokes augment-scoped advice; buying one component does not revoke objective advice, which depends on who is alive rather than on gold.

**Table 4.** Dependencies and lifetimes per coaching stage.

| Stage | Depends on | TTL |
|---|---|---|
| `opening` (six-item plan) | champion **and owned augments** | 10 min |
| `augment` (three-way choice) | champion, owned, current offers | 90 s |
| `postAugment` | champion, owned | 90 s |
| `death` / `purchase` | champion, phase, level band, completed items, owned, enemy items | 45 s |
| `objective` / `turret` / `ace` / `coordination` | champion, phase, alive allies, alive enemies | 45 s |

## 6. Case Study: Augments That Name Items

Some augments reference a specific item. 钢化你心 ("Steel Your Heart") requires holding 心之钢 (Heartsteel) and stacking it; 命运之武器 applies to your first legendary item; 易损 rewards critical-strike items.

In a real recorded match the player confirmed 钢化你心 at roughly three minutes. The six-item build plan for that match read: *Mercury's Treads → Force of Nature → Randuin's → Frozen Heart → Spirit Visage → Stoneplate*. No Heartsteel anywhere — while a later single recommendation told the player to combine Giant's Belt into Heartsteel to unlock the augment. Two subsystems disagreed, because the opening plan declared its dependency as the champion alone:

```
if (stage === 'opening') return { champion: me.champion };   // augments ignored
```

With a ten-minute TTL, that stale plan stayed "valid" for the rest of the early game. The fix has three parts: opening plans now depend on owned augments; a hard prompt rule requires that a confirmed augment which names an item forces that item into the build order with the reason stated; and `postAugment` now re-derives the remaining two or three items at the moment the augment becomes known. An audit of the last eight matches found four of the observed augments constrain item choice, so this is not an edge case.

## 7. Presentation and Voice

The overlay is WPF hosted by PowerShell 5.1 in four display modes. The narrow mode keeps a live subtitle strip rather than collapsing to a dot, so the player does not lose the last sentence. Subtitle splitting respects

semantics: punctuation never starts a line, number tokens such as `No.22`, `53.5%` and `15/6/16` stay whole, and at most two rows are visible.

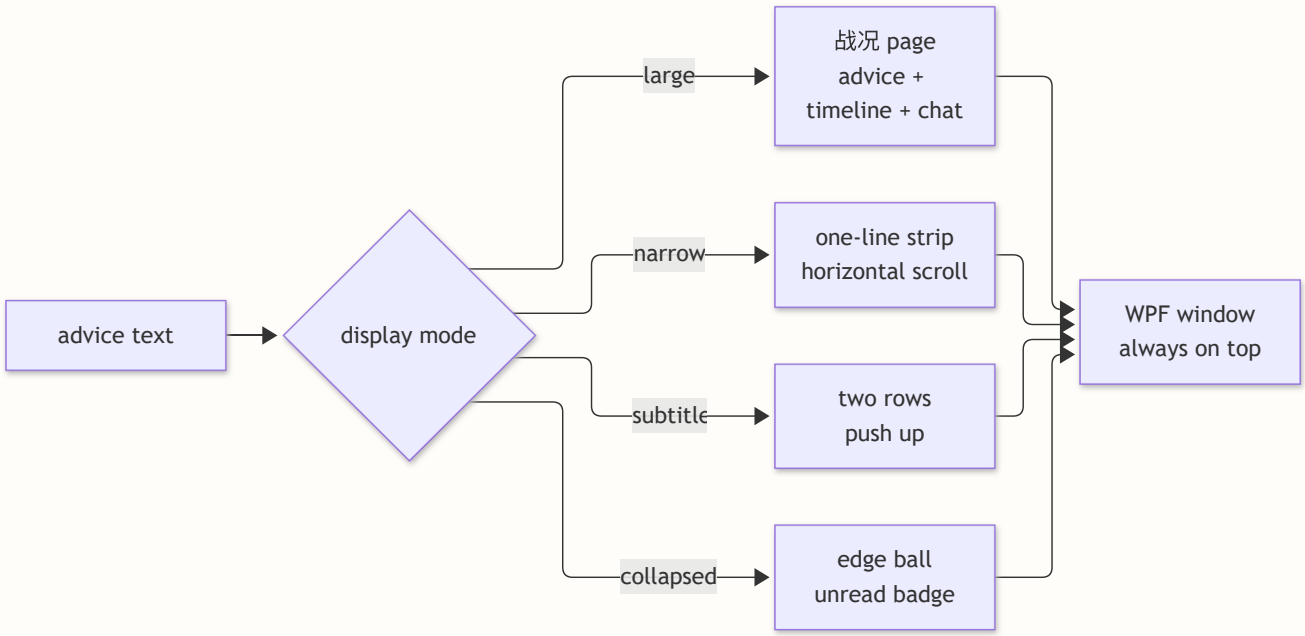



**Figure 5.** Display modes and what each one preserves. Only the collapsed ball drops the last sentence, keeping an unread badge instead.

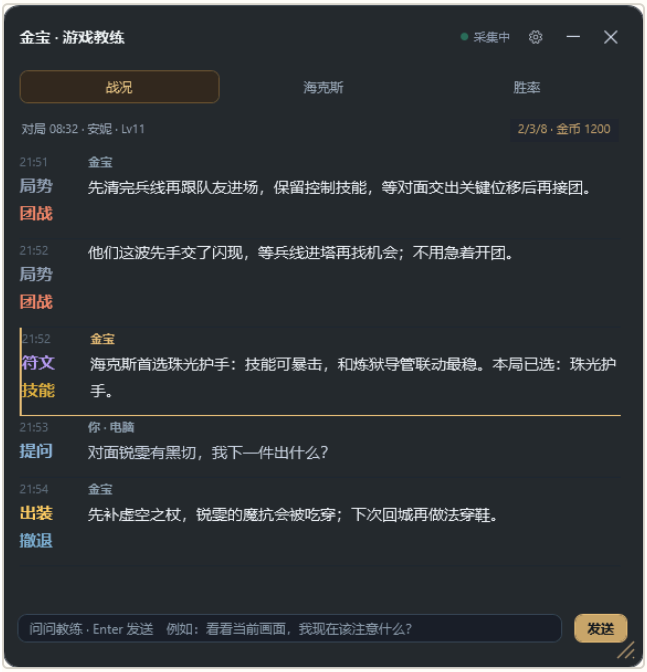



**Figure 6.** A real advice stream during a match: death, coordination and build items, each one or two sentences.

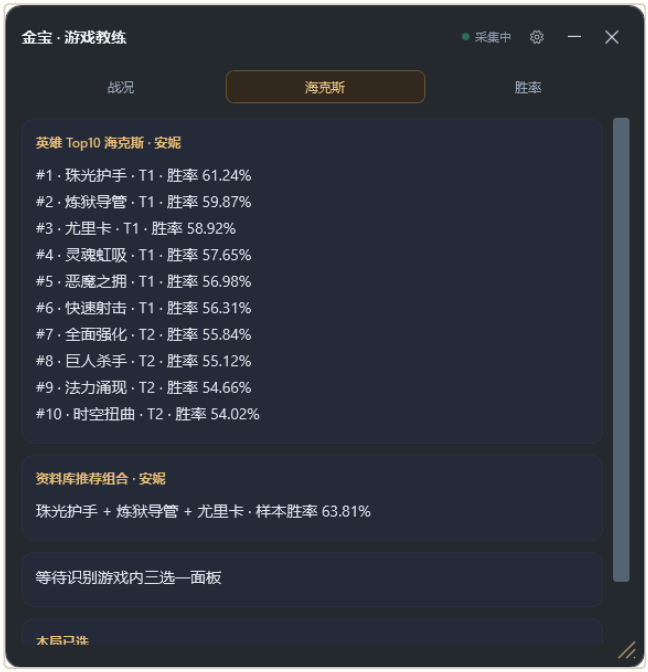



**Figure 7.** Augment ranking for the current champion, with owned picks taken into account.

The speech queue expires *only queued* items; once audio starts it finishes. Every transition (`queued`, `started`, `completed`, `interrupted`, `discarded`) is appended to a local audit log together with the original and spoken text, so display-versus-speech drift is auditable rather than argued about.

## 8. Failure Modes and Lessons

Three incidents shaped the engineering process more than any feature.

**Table 5.** Failures, their observable symptom, and the rule adopted afterwards.

| Failure | Symptom | Rule adopted |
|---|---|---|
| JavaScript comment in a PowerShell file | Overlay crashed on every collapse; supervisor restarted it, which looked like "the window disappears and comes back" | `.ps1` files are UTF-8 with BOM and use only `#` comments; every unhandled exception logs source, line and code |
| Log statement inserted between `elseif` and `else` | `CommandNotFoundException: else` at a known line | Instrument first, then edit; the next bug after a fix is usually the fix itself |
| Display string treated as data | `(view.events \|\| []).map(…)` threw; `try/catch` swallowed it; **three consecutive matches produced no memory** | Never silence a pipeline: log the skip reason or fail loudly |

The third case is the most instructive. The projection boundary is real — `events` is text for rendering, not an array — and the recovery path is now the same one used for everything else: the recorder reads the match's own journal for authoritative `GameEnd.Result`, final KDA, level, duration and multikill streaks, and a replay script restored the six lost matches from those journals.

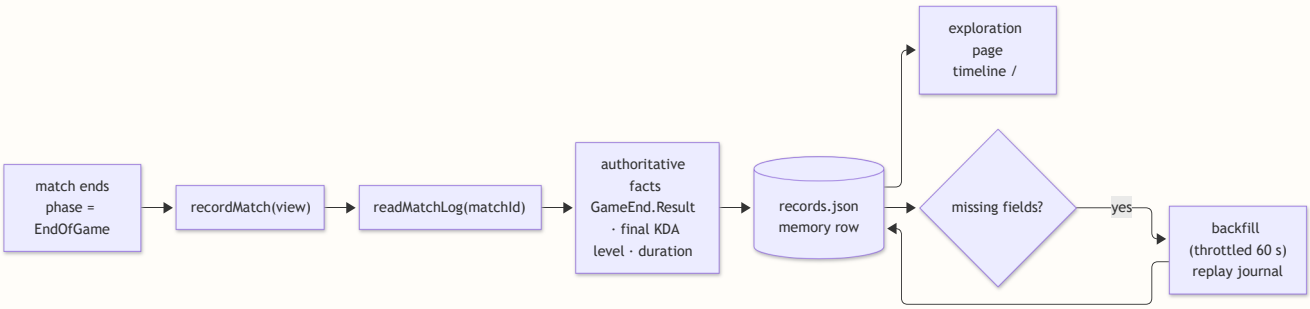
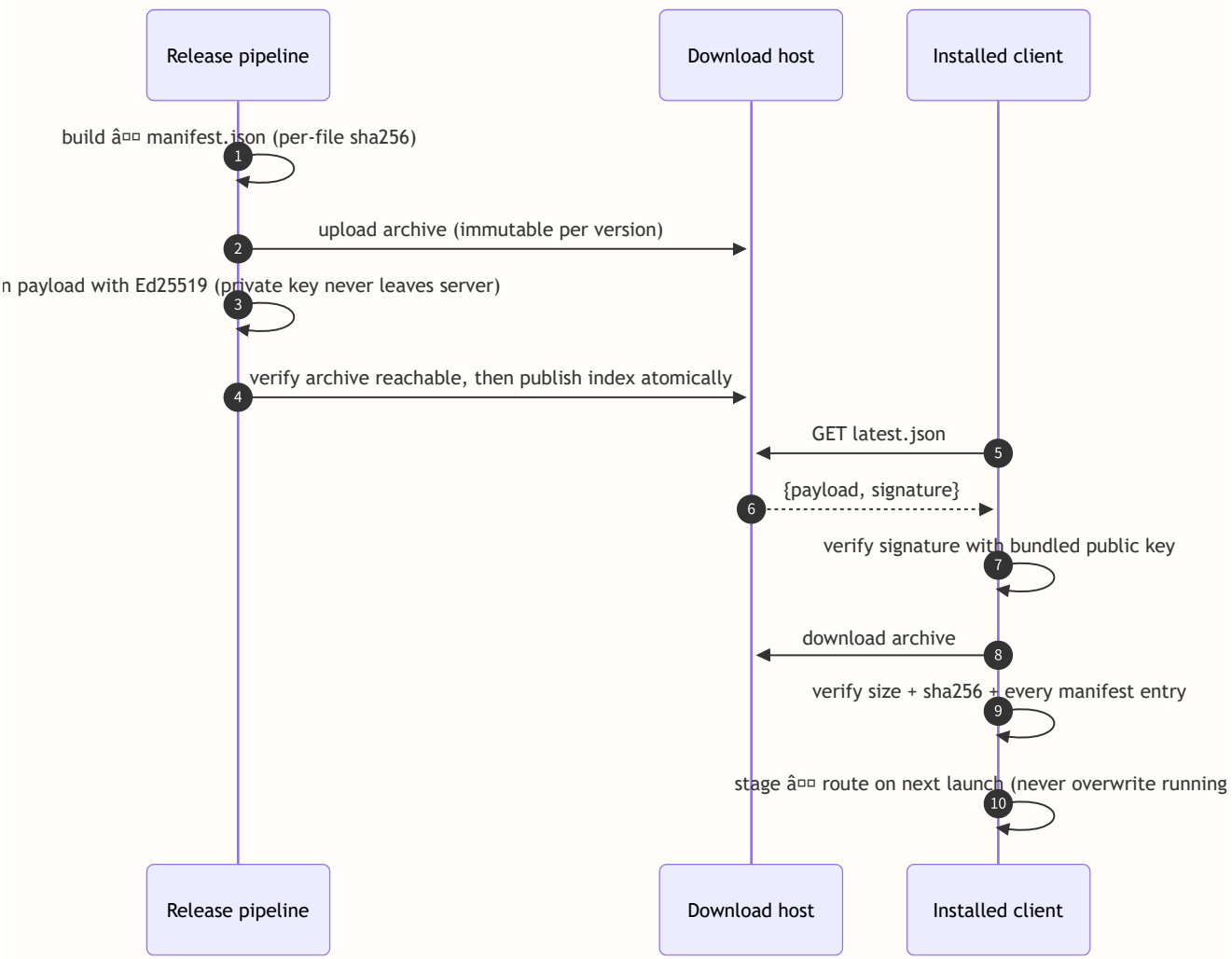



**Figure 8.** The memory pipeline that replaced the broken path: authoritative values come from the journal, and a throttled backfill repairs records that are missing fields.

## 9. Privacy, Credentials and Update Integrity

Match data, memories, preferences and diagnostics live under the user's profile directory and are never uploaded; there is no leaderboard. Credentials are DPAPI-encrypted per Windows user, and bring-your-own-key model secrets never reach our servers. Diagnostics exports are redacted: no chat text, player names, credentials or screenshots.




**9.** Signed update pipeline. The index is published only after the archive is verified reachable, so a client can never accept an index pointing g artifact.

**Table 6.** Update integrity controls.

| Control | Implementation |
|---|---|
| Tamper resistance | Ed25519 signature over the exact payload bytes; fixed official origin; public key shipped in the client |
| Rollback safety | Immutable version paths; atomic index publication after archive verification |
| Partial download | Size plus per-file hash verification against the package manifest before staging |
| Failure policy | Any mismatch keeps the old version running; the new version stages and routes on next launch |

## 10. Evaluation and Limitations

**Table 7.** What has been verified in this release cycle.

| Area | Evidence |
|---|---|
| Advice validity | Unit tests over dependency fingerprints: a later augment invalidates, a component purchase does not |
| Augment-aware builds | Prompt assertions plus offline replay of a real match journal; four item-constraining augments enumerated |
| Memory recording | Journal replay produced KDA 32/11/21, win, level 18, 1247 s, triple, quadra and ace |
| Speech audit | 116 recorded transitions in one match, original and spoken text diffable |
| Overlay stability | Zero unhandled UI exceptions across an evening of collapse and resize cycles |

**Known limitations.** Augment identity is OCR-derived; ambiguous rounds fall back to user confirmation, and recall is better than precision by design. League of Legends is the only deeply adapted title — other games receive window observation with explicitly unverified identity. The model can still produce advice that is technically true but operationally vague; the rules reduce this without eliminating it. Some sandbox-dependent tests are environment-sensitive and excluded from the default run. Finally, advice quality has not been validated against a control group: "good advice" remains a judgement call, not a measurement.

## 11. Non-Goals

- Predicting match outcomes or assigning probabilities to a live game.
- Automating input, reading game memory, or injecting into the client.
- A cloud dashboard or leaderboard. The interesting data stays on the machine that produced it.

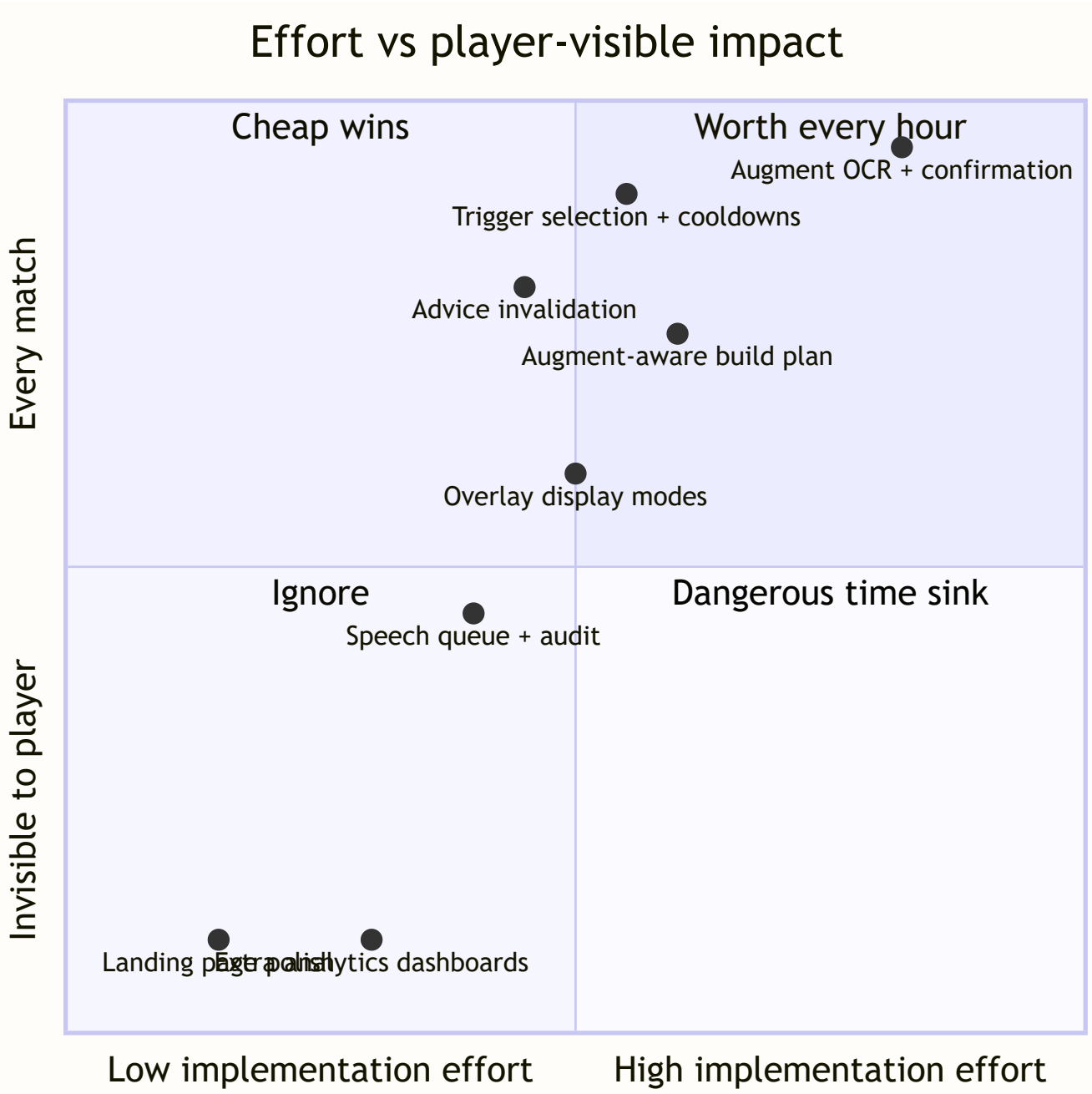



**Figure 10.** Where effort actually pays. The upper-left cluster — cheap to describe, expensive to get right — is the reason this project exists.

## 12. Data Sources

League client log files and the local LCU HTTP API — the same public surface used by overlay-class tools. Community and third-party data (augment names, champion statistics) is fetched through a separate versioned index and never embedded as authoritative truth; when a value is missing, the advice says so rather than inventing it.

## References

1. League of Legends Live Client Data API. Riot Games developer documentation.
2. League Client Update (LCU) API — local REST surface exposed by the desktop client.

3. B. Josefsson, I. Liusvaara. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. RFC 8032, 2017.
4. Microsoft. *Windows Data Protection API (DPAPI)* — CryptProtectData / CryptUnprotectData.
5. Microsoft. *Windows Presentation Foundation* — retained-mode UI framework used by the overlay.
6. Node.js Foundation. *Node.js 22 LTS runtime* — the bundled runtime of the standalone package.
7. Mermaid diagram tooling — figures in this report are rendered from the repository's Mermaid sources.
8. Source repository: github.com/YuanCao1996/dsh-game-jinbao (`docs/ARCHITECTURE.md`, `docs/TECHNICAL_REPORT.md`).

---